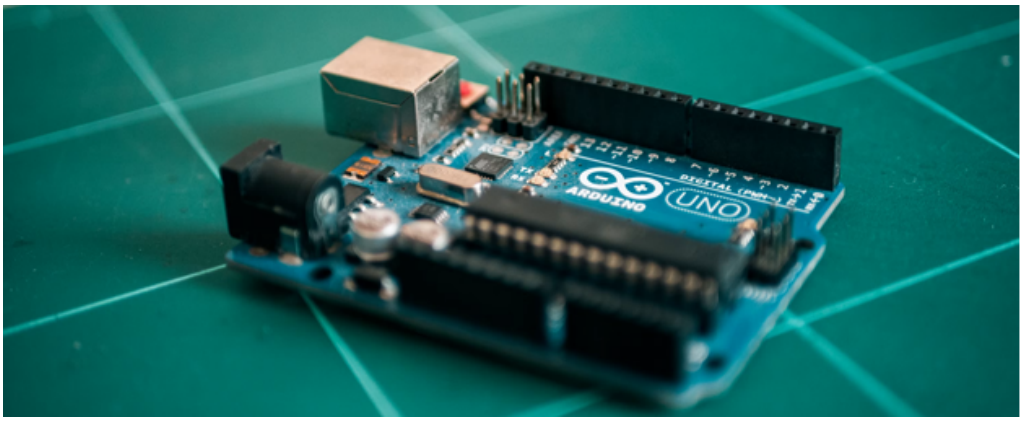
UNIVERSITÀ DI CAMERINO

Scuola di Scienze e Tecnologie – Sezione Informatica

Corso di Architettura degli Elaboratori – Modulo Laboratorio



CENTRALINA CONTROLLO ACCESSI

Sistema digitale di autenticazione a due fattori (PIN + OTP)

Docente:
Prof. Massimo Trojani

Candidato:
Alessandro Virivè
Matricola: 130052
Email: alessandro.virive@studenti.unicam.it

Link progetto simulatore Wokwi:

`https://wokwi.com/projects/478132265244059649`

Anno Accademico 2026/2027

Indice

1 Analisi **2**
1.1 Testo assegnato . . . 2
1.2 Analisi del problema . . . 3
1.2.1 Prima analisi del problema . . . 3
1.2.2 Analisi dell'algoritmo di autenticazione a due fattori e verifica booleana in serie 4
1.2.3 Componente CLOCK . . . 6
1.2.4 Componente SENSORI . . . 6
1.2.5 Componente CONTROLLO . . . 7
1.2.6 Componente GENERATORE_OTP . . . 8
1.2.7 Componente VISIVO . . . 9
1.2.8 Flusso logico complessivo . . . 10
1.3 Analisi della soluzione adottata . . . 11
1.3.1 Scelte comuni e architettura del sistema . . . 11
1.3.2 Modulo CLOCK . . . 12
1.3.3 Modulo SENSORI (PIR e Tastiera) . . . 12
1.3.4 Modulo CONTROLLO (FSM a 7 stati) . . . 14
1.3.5 Modulo GENERATORE_OTP, UART e Modulo VISIVO . . . 15
1.3.6 Sistema complessivo . . . 16

2 Sintesi **17**
2.1 Sintesi Modulo CLOCK e Base Tempi . . . 17
2.2 Sintesi Modulo SENSORI (PIR e Tastiera) . . . 18
2.3 Sintesi Modulo CONTROLLO . . . 20
2.4 Sintesi Modulo GENERATORE_OTP, UART e Modulo VISIVO . . . 23
2.5 Analisi di livello MV2 e qualità del codice assembly . . . 24
2.5.1 1. Ottimizzazione delle istruzioni di I/O (`SBI`, `CBI`, `SBIC`, `SBIS`) . . . 24
2.5.2 2. Effetto architetturale di `static inline` . . . 24
2.5.3 3. Impatto del qualificatore `volatile` e atomicità a 8 bit . . . 24
2.5.4 4. Costo computazionale delle ISR . . . 24

3 Sistema complessivo **25**
3.1 Manuale di utilizzo . . . 26
3.2 Considerazioni finali . . . 27

1 Analisi

1.1 Testo assegnato

Realizzare un sistema di controllo accessi costituito dalle seguenti periferiche:

- a) due display a 7 segmenti da 4+2 cifre ciascuno (`wokwi-7segment`)
- b) un keypad a 3 colonne e 4 righe `{"attrs": {"columns":"3"}}` (`wokwi-membrane-keypad`)
- c) un rilevatore di movimento (`wokwi-pir-motion-sensor`)

Il sistema quando rileva un movimento si pone in modalità operativa ovvero:

1. Premendo il tasto "`*`" il sistema aspetta il codice personale da 6 cifre.
2. Riconosciuto il codice personale come autorizzato, invia un SMS con un codice OTP che dovrà essere inserito dal soggetto, preceduto dal tasto "`#`".
3. Se il codice OTP inserito è corretto e viene inserito entro 10 secondi, il sistema attiva l'apertura della cassaforte, altrimenti attiva il sistema di allarme.

Deve essere possibile interrompere il processo di accesso digitando in sequenza "`*`" e "`#`". In questa versione, il codice OTP verrà inviato e visualizzato sulla console seriale.

La progettazione dovrà essere svolta secondo il paradigma:

- Minori risorse impiegate per la macchina;
- Maggior sforzo per la progettazione e sviluppo.

In particolare:

- a) I componenti HW-SW dovranno essere "Automi" indipendenti che comunicano con variabili globali;
- b) Le cadenze temporali (ritardi ecc.) dovranno essere governati dall'ISR del clock;
- c) La progettazione delle ISR dovrà garantire tempi di servizio compatibili con le risposte al tempo reale del sistema;
- d) La gestione degli I/O deve essere diretta, senza usare le funzioni di "Arduino" o librerie esterne. Ad esempio: il keypad può essere gestito in foreground su loop ma senza usare funzioni di libreria; le funzioni di visualizzazione sui display dovranno essere indipendenti e temporizzate tramite Interrupt del clock.

1.2 Analisi del problema

1.2.1 Prima analisi del problema

Dalla scomposizione analitica del testo di specifica si ricavano i problemi logico-funzionali elementari da risolvere, isolando la logica computazionale di controllo dalle interfacce fisiche esterne:

- **a) Rilevamento dell'evento di attivazione ambientale:** Tradurre la variazione fisica di movimento nell'area protetta in un segnale logico binario $M \in \{0, 1\}$ capace di commutare la macchina dallo stato inattivo iniziale (in cui i visualizzatori sono spenti e la tastiera è inibita) allo stato operativo di attesa comando.
- **b) Discretizzazione, filtraggio temporale e codifica binaria della tastiera:** Acquisire lo stato di chiusura dei 12 interruttori disposti su una griglia di 4 righe per 3 colonne, rimuovere le oscillazioni spurie di commutazione meccanica (rimbalzi di chiusura e di apertura) mediante un filtro di stabilità su N campioni consecutivi e codificare univocamente ciascuno dei 12 simboli (cifre decimali 0..9 e delimitatori `*`, `#`) più la condizione di riposo in una parola binaria minima a 4 bit $T \in [0, 15]$.
- **c) Verifica sequenziale a due fattori in serie (PIN statico e OTP dinamico):** Validare una prima sequenza di 6 cifre decimali confrontandola cifra per cifra con una sequenza costante cablata (PIN_RIF[0..5]) mediante un singolo accumulatore booleano (V), senza allocare vettori intermedi di memorizzazione dell'input; in caso di esito positivo ($V = 1$), generare una seconda sequenza pseudo-casuale di 6 cifre (OTP[0..5]), trasmetterla su linea seriale e validarne il reinserimento da parte dell'operatore entro un intervallo temporale tassativo di 10 secondi.
- **d) Intercettazione prioritaria della sequenza di annullamento (`*#`):** Riconoscere in qualsiasi stato intermedio del flusso operativo la digitazione consecutiva del simbolo `*` (10_{10}) seguito immediatamente dal simbolo `#` (11_{10}), forzando l'azzeramento dei contatori, l'arresto del conteggio temporale, lo spegnimento dei visualizzatori e il ritorno immediato allo stato dormiente iniziale.
- **e) Scansione ciclica a divisione di tempo dell'interfaccia visiva:** Proiettare il contenuto di un vettore simbolico di 6 elementi sui 6 moduli a 7 segmenti attivando ciclicamente un solo elemento luminoso ad ogni passo della base tempi, riducendo il numero di linee di pilotaggio e garantendo la persistenza ottica dell'immagine senza arresti bloccanti nel flusso principale.

I punti sopra definiti identificano le 5 componenti funzionali del sistema oggetto di analisi dettagliata:

1. Un componente **CLOCK** che genera gli impulsi periodici di sincronizzazione ($\Delta t = 1$ ms) e gestisce il contatore decrescente in secondi per il vincolo temporale dell'OTP.
2. Un componente **SENSORI** che campiona il rilevatore di movimento e la matrice di contatti, applicando l'algoritmo di antirimbalzo e fornendo in uscita i segnali binari stabilizzati M e T.
3. Un componente **CONTROLLO** che racchiude la Macchina a Stati Finiti (FSM) principale per il coordinamento delle fasi di autenticazione, la comparazione logica in serie e il comando degli stati di apertura e allarme.
4. Un componente **GENERATORE_OTP** che trasforma algebricamente un seme numerico ad alta frequenza in una sequenza di 6 cifre decimali pseudo-casuali e ne esegue la serializzazione in uscita.

5. Un componente **VISIVO** che trascodifica i simboli numerici e di stato nelle corrispondenti configurazioni a 7 segmenti e ne governa la scansione ciclica sulle 6 cifre.

1.2.2 Analisi dell'algoritmo di autenticazione a due fattori e verifica booleana in serie

La procedura di controllo accessi subordina l'attivazione del segnale di apertura al superamento in cascata di due verifiche su stringhe numeriche di lunghezza fissa $L = 6$. Per minimizzare il numero di bit di memoria impiegati dalla macchina già in fase di analisi, la comparazione tra la sequenza digitata dall'operatore e la sequenza attesa non viene effettuata memorizzando tutte le 6 cifre in un vettore di ingresso per poi confrontarle a fine inserimento, ma viene calcolata **in serie ad ogni singola cifra acquisita** mediante un unico bit accumulatore di validità $V \in \{0, 1\}$.

Il flusso logico universale dell'algoritmo è definito dalla seguente sequenza di passi imperativi:

1. **Condizione iniziale di riposo:** Poni lo stato del sistema a `STANDBY`, poni il comando del timer a abilita_timer $= 0$ e imposta tutte le 6 celle del vettore visivo al simbolo `VUOTO` (11_{10}, segmenti spenti).

2. **Transizione di risveglio:** Quando il segnale del sensore di movimento assume valore $M == 1$, azzera $M = 0$, imposta tutte le 6 celle del vettore visivo al simbolo `TRATTINO` (10_{10}, visualizzazione `---`), poni la memoria del tasto precedente a ultimo_tasto $= 15_{10}$ (`NESSUNO`) e transita nello stato `ATTESA_ASTERISCO`.

3. **Innesco acquisizione primo fattore (PIN):** Nello stato `ATTESA_ASTERISCO`, preleva il simbolo validato T dal componente SENSORI:

   - Se $T == 10_{10}$ (simbolo `*`), poni il contatore di posizione a indice $= 0$, inizializza l'accumulatore di validità a $V = 1$, imposta il vettore visivo a `---` e transita nello stato `INSERIMENTO_PIN`.
   - Se $T \neq 10_{10}$, mantieni lo stato `ATTESA_ASTERISCO`.

4. **Acquisizione e comparazione in serie del PIN:** Nello stato `INSERIMENTO_PIN`, ad ogni nuovo simbolo T prelevato:

   - Se $T \leq 9$ (cifra decimale valida):
     (a) Scrivi il valore T nella cella celle_display[indice].
     (b) Calcola il nuovo valore dell'accumulatore booleano di validità:

     $$V \leftarrow V \wedge \big(T == \text{PIN_RIF}[\text{indice}]\big)$$

     (c) Incrementa il contatore di posizione: indice $\leftarrow$ indice $+ 1$.
   - Se indice $== 6$ (sesta cifra acquisita), valuta il bit V:
     – Se $V == 0$ (almeno una cifra errata), imposta tutte le 6 celle del vettore visivo alla cifra 0 (`000000`) e transita nello stato terminale `ALLARME`.
     – Se $V == 1$ (sequenza PIN interamente corretta), preleva il valore corrente del contatore a 16 bit `seme_casuale`, calcola le 6 cifre decimali OTP[0..5], inviale sul canale seriale, imposta il contatore temporale a secondi_residui $= 10$, poni fine_tempo $= 0$, attiva il conteggio (abilita_timer $= 1$), imposta il vettore visivo a `---` e transita nello stato `ATTESA_CANCELLETTO`.

5. **Attesa delimitatore secondo fattore (#) sotto vincolo temporale:** Nello stato `ATTESA_CANCELLETTO`:

   - Se il segnale di scadenza temporale assume valore fine_tempo $== 1$, imposta il vettore visivo a `000000` e transita immediatamente nello stato `ALLARME`.

- Se fine_tempo == 0 e viene acquisito il simbolo $T == 11_{10}$ (`#`), poni indice = 0, poni $V = 1$, imposta il vettore visivo a `---` e transita nello stato `INSERIMENTO_OTP` (lasciando attivo il decremento di `secondi_residui`).

6. **Acquisizione temporizzata dell'OTP e verdetto finale:** Nello stato `INSERIMENTO_OTP`:
   - Se in qualsiasi ciclo di scansione fine_tempo == 1, imposta il vettore visivo a `000000` e transita immediatamente in `ALLARME`.
   - Se fine_tempo == 0 e viene acquisita una cifra $T \leq 9$:
     (a) Scrivi T in celle_display[indice].
     (b) Aggiorna la validità: $V \leftarrow V \wedge \big(T == \text{OTP[indice]}\big)$.
     (c) Incrementa il contatore: indice $\leftarrow$ indice $+ 1$.
   - Quando indice == 6 con fine_tempo == 0, arresta il temporizzatore (abilita_timer = 0) e testa il bit V:
     – Se $V == 1$, imposta le 6 celle del vettore visivo alla cifra 1 (`111111`, segnale di apertura cassaforte) e transita nello stato terminale `APERTO`.
     – Se $V == 0$, imposta le 6 celle del vettore visivo alla cifra 0 (`000000`) e transita nello stato terminale `ALLARME`.

7. **Riconoscitore prioritario della sequenza d'arresto (`*#`):** In tutti gli stati operativi non terminali (`ATTESA_ASTERISCO`, `INSERIMENTO_PIN`, `ATTESA_CANCELLETTO`, `INSERIMENTO_OTP`), ad ogni nuovo simbolo valido $T \neq 15_{10}$, testa preliminarmente la condizione booleana:

$$\big(\text{ultimo_tasto} == 10_{10}\big) \wedge \big(T == 11_{10}\big)$$

   Se la condizione è vera, poni abilita_timer = 0, fine_tempo = 0, ultimo_tasto = 15_{10}, imposta il vettore visivo a `VUOTO` (11_{10}) e forza la transizione immediata nello stato `STANDBY`. Se la condizione è falsa, aggiorna ultimo_tasto $\leftarrow T$ e procedi con la valutazione ordinaria dello stato corrente.

1.2.3 Componente CLOCK

Il componente **CLOCK** misura il trascorrere del tempo reale e fornisce agli automi del sistema le primitive di sincronizzazione e di conteggio alla rovescia. Prescindendo dall'implementazione circuitale dell'oscillatore, il componente mantiene:

- Un segnale impulsivo `tick_ms` che assume valore logico 1 ad ogni intervallo elementare costante $\Delta t = 1$ ms (1000 Hz), scandendo il passo discreto di evoluzione degli automi.
- Un divisore di frequenza interno (accumulatore modulo 1000, variabile nell'intervallo 0..999 $\rightarrow \lceil \log_2 1000 \rceil = 10$ bit) attivo quando il segnale di ingresso `abilita_timer` vale 1. Ad ogni raggiungimento di 1000 impulsi da 1 ms (1 secondo esatto), l'accumulatore si azzera e decrementa di un'unità il registro `secondi_residui` (inizializzato a 10). Quando `secondi_residui` raggiunge il valore 0, il componente azzera `abilita_timer` e pone a 1 il segnale di uscita `fine_tempo`.

Questa scomposizione a due stadi riduce la grandezza condivisa del timeout dall'intervallo 0..10 000 ms ($\lceil \log_2 10001 \rceil = 14$ bit) al solo intervallo 0..10 s, rappresentabile con soli $\lceil \log_2 11 \rceil = \mathbf{4}$ bit. Il modulo CLOCK scambia con il sistema i seguenti segnali:

Nome	Dimensione (bit)	Descrizione
`tick_ms`	1	Impulso di cadenza temporale (1 = trascorso 1 ms, 0 = attesa)
`abilita_timer`	1	Comando di controllo del timer OTP (1 = conta, 0 = fermo e azzerato)
`secondi_residui`	4	Contatore dei secondi rimanenti prima della scadenza (intervallo 0–10)
`fine_tempo`	1	Segnale di evento scadenza dei 10 secondi (1 = tempo scaduto, 0 = valido)

1.2.4 Componente SENSORI

Il componente **SENSORI** converte le grandezze fisiche d'ingresso (radiazione infrarossa del corpo umano e chiusura meccanica dei tasti) in informazioni digitali filtrate e codificate in binario:

1. **Interfaccia rilevatore di movimento:** Sul fronte di salita ($0 \rightarrow 1$) del segnale fisico proveniente dal sensore volumetrico, il componente posiziona a 1 il segnale booleano di uscita M (`movimento`), mantenendolo alto finché non viene azzerato dal componente CONTROLLO.
2. **Scansione matriciale e automa di antirimbalzo (Debounce):** La tastiera è costituita da 12 interruttori momentanei posti agli incroci di 4 righe ($r \in [0, 3]$) e 3 colonne ($c \in [0, 2]$). Ad ogni impulso `tick_ms` (1 ms), il componente attiva in sequenza una riga alla volta e rileva lo stato delle 3 colonne, ricavando la coordinata posizionale grezza $C_{\text{raw}} = 3r + c \in [0, 11]$ (oppure il codice 15 se nessun contatto è chiuso). Poiché le lamine metalliche di un pulsante generano rimbalzi transitori per diversi millisecondi dopo l'impatto, una Macchina a Stati Finiti interna a 4 stati ($\lceil \log_2 4 \rceil = \mathbf{2}$ bit) filtra il segnale:
   - `RIPOSO` (0): Se $C_{\text{raw}} \neq 15$, memorizza la coordinata in coord_stabile $\leftarrow C_{\text{raw}}$, azzera il contatore contatore_stabile $= 0$ e transita in `VERIFICA_PRESSIONE`.
   - `VERIFICA_PRESSIONE` (1): Se $C_{\text{raw}} ==$ coord_stabile, incrementa `contatore_stabile`; al raggiungimento di 16 campioni consecutivi (16 ms, $\lceil \log_2 17 \rceil = \mathbf{5}$ bit), traduce coord_stabile nel codice binario del simbolo $T \in [0, 11]$ e transita in `ATTESA_RILASCIO`. Se invece $C_{\text{raw}} \neq$ coord_stabile (rimbalzo spurio), ritorna in `RIPOSO`.

- `ATTESA_RILASCIO` (2): Mantiene bloccate nuove emissioni finché il tasto resta premuto (evitando letture multiple per una singola pressione). Appena rileva $C_{\text{raw}} == 15$, azzera contatore_stabile $= 0$ e transita in `VERIFICA_RILASCIO`.
- `VERIFICA_RILASCIO` (3): Incrementa `contatore_stabile` ad ogni millisecondo in cui $C_{\text{raw}} == 15$; raggiunti 16 campioni consecutivi di apertura stabile (16 ms), ritorna in `RIPOSO`; se rileva un rimbalzo di chiusura ($C_{\text{raw}} \neq 15$), ritorna in `ATTESA_RILASCIO`.

I 12 simboli della tastiera più il valore sentinella di assenza tasto costituiscono 13 configurazioni distinte, che richiedono $\lceil \log_2 13 \rceil = \mathbf{4}$ bit ($2^4 = 16 \geq 13$). Il componente adotta una codifica binaria diretta pura ($0000_2..1001_2$ per le cifre 0..9, $1010_2 = 10$ per `*`, $1011_2 = 11$ per `#`, $1111_2 = 15$ per nessun tasto), eliminando qualsiasi ricorso a codifiche testuali ASCII a 8 bit:

Nome	Dimensione (bit)	Descrizione
`M (Movimento)`	1	Segnale di evento presenza rilevata (1 = movimento, 0 = quiete)
`T (Tasto)`	4	Parola binaria del tasto validato (0..9 cifre, 10 = `*`, 11 = `#`, 15 = nullo)
`coord_stabile`	4	Coordinata posizionale ($3r + c$) in corso di verifica (intervallo 0–11, 15)
`stato_filtro`	2	Stato interno della FSM di antirimbalzo (4 stati: intervallo 0–3)
`contatore_stabile`	5	Contatore dei campioni identici consecutivi a 1 ms (intervallo 0–16)

1.2.5 Componente CONTROLLO

Il componente **CONTROLLO** costituisce l'unità decisionale centrale del sistema, modellata come una Macchina a Stati Finiti (FSM) a 7 stati ($\lceil \log_2 7 \rceil = \mathbf{3}$ bit, poiché $2^3 = 8 \geq 7$) che elabora i segnali provenienti da SENSORI (M, T) e da CLOCK (`fine_tempo`), governando il temporizzatore (`abilita_timer`), l'innesco del componente GENERATORE_OTP e l'aggiornamento del componente VISIVO (`celle_display`).

Gli stati logici della FSM sono definiti come segue:

- `STANDBY` **(0):** Stato dormiente di partenza. Tutte le celle visive sono poste a 11 (`VUOTO`) e gli ingressi T sono ignorati. Commuta in `ATTESA_ASTERISCO` impostando le celle a 10 (`---`) esclusivamente quando $M == 1$.
- `ATTESA_ASTERISCO` **(1):** Stato di attesa del delimitatore di inizio. Scarta qualsiasi simbolo $T \neq 10$; quando $T == 10$ (`*`), azzera indice $= 0$, pone $V = 1$, imposta le celle a `---` e commuta in `INSERIMENTO_PIN`.
- `INSERIMENTO_PIN` **(2):** Stato di acquisizione e verifica del PIN. Ad ogni cifra $T \in [0, 9]$, copia T in celle_display[indice], aggiorna $V \leftarrow V \wedge (T ==$ PIN_RIF[indice]) e incrementa indice ($\lceil \log_2 7 \rceil = \mathbf{3}$ bit). Quando indice $== 6$: se $V == 1$, innesca il calcolo e la trasmissione dell'OTP, imposta secondi_residui $= 10$, pone abilita_timer $= 1$, imposta le celle a `---` e commuta in `ATTESA_CANCELLETTO`; se $V == 0$, imposta le celle a 0 (`000000`) e commuta in `ALLARME`.
- `ATTESA_CANCELLETTO` **(3):** Stato di attesa conferma OTP. Se fine_tempo $== 1$, imposta le celle a `000000` e commuta in `ALLARME`. Se fine_tempo $== 0$ e $T == 11$ (`#`), azzera indice $= 0$, pone $V = 1$, imposta le celle a `---` e commuta in `INSERIMENTO_OTP`.

- `INSERIMENTO_OTP` **(4):** Stato di acquisizione temporizzata del codice dinamico. Se fine_tempo == 1, imposta le celle a `000000` e commuta in `ALLARME`. Ad ogni cifra $T \in [0, 9]$ con fine_tempo == 0, scrive T in celle_display[indice], aggiorna $V \leftarrow V \wedge (T == \text{OTP[indice]})$ e incrementa indice. Quando indice == 6, arresta il timer (abilita_timer = 0): se $V == 1$ imposta le celle a 1 (`111111`) e commuta in `APERTO`; se $V == 0$ imposta le celle a 0 (`000000`) e commuta in `ALLARME`.
- `ALLARME` **(5):** Stato terminale di blocco sicurezza (visualizzazione permanente `000000`).
- `APERTO` **(6):** Stato terminale di cassaforte sbloccata (visualizzazione permanente `111111`).

Nome	Dimensione (bit)	Descrizione
`stato`	3	Stato corrente della FSM centrale (7 stati: intervallo 0–6)
`indice`	3	Contatore delle cifre decimali acquisite nella fase attiva (intervallo 0–6)
`V (valido)`	1	Accumulatore booleano della verifica cifra a cifra (1 = valido, 0 = errato)
`ultimo_tasto`	4	Memoria dell'ultimo simbolo ricevuto per il pattern di reset `*#` (0–11, 15)
`PIN_RIF[0..5]`	24	Sequenza costante di 6 parole da 4 bit del codice personale (`1, 2, 3, 4, 5, 6`)

1.2.6 Componente GENERATORE_OTP

Il componente **GENERATORE_OTP** sintetizza la sequenza numerica temporanea OTP[0..5] e la invia sul canale di comunicazione seriale verso l'operatore. Poiché una macchina digitale deterministica richiede una sorgente di entropia esterna per produrre sequenze non predicibili, il componente mantiene un registro a 16 bit (`seme_casuale`) incrementato ininterrottamente ad ogni ciclo macchina. Nell'istante (asincrono e dipendente dal tempo di reazione umano) in cui viene validata la sesta cifra del PIN, il valore istantaneo S = seme_casuale viene sottoposto per 6 iterazioni successive alla trasformazione algebrica a scorrimento e XOR bit a bit (*Xorshift* a 16 bit):

$$S \leftarrow S \oplus (S \ll 7), \qquad S \leftarrow S \oplus (S \gg 9), \qquad S \leftarrow S \oplus (S \ll 8)$$

Ad ogni iterazione $i \in [0, 5]$, vengono estratti i 4 bit meno significativi $d_i = S \wedge 0\text{x}0\text{F} \in [0, 15]$; se $d_i > 9$, viene sottratta la costante 6 ($d_i \leftarrow d_i - 6$), mappando il valore nell'insieme decimale BCD $[0, 9]$ senza ricorrere a divisioni intere.

Isolamento della codifica binaria interna dal protocollo del terminale esterno: Internamente alla macchina, le 6 cifre generate sono memorizzate nel vettore OTP[0..5] come numeri binari puri a 4 bit (0..9), omogenei con la codifica di T prodotta dal componente SENSORI. La somma della costante 48_{10} (codice ASCII del carattere `'0'`) viene eseguita esclusivamente sul dato in uscita verso il trasmettitore seriale per soddisfare il vincolo di visualizzazione leggibile sul terminale testuale umano. In un'interconnessione diretta Macchina-Macchina (M2M), tale traslazione viene omessa trasmettendo direttamente la parola binaria a 4 bit.

Nome	Dimensione (bit)	Descrizione
`seme_casuale`	16	Contatore ad incremento libero per il campionamento dell'entropia (1–65535)
`OTP[0..5]`	24	Vettore di 6 parole binarie da 4 bit contenente le cifre OTP generate (0–9)

1.2.7 Componente VISIVO

Il componente **VISIVO** governa la rappresentazione ottica dello stato e delle cifre sui 6 elementi a 7 segmenti (A, B, C, D, E, F, G). Riceve in ingresso dal componente CONTROLLO il vettore `celle_display[0..5]` costituito da 6 parole da **4** bit ($\lceil \log_2 12 \rceil = 4$ bit: valori 0..9 per i caratteri numerici, 10 per il trattino centrale - sul solo segmento G, 11 per tutti i segmenti spenti). Ad ogni impulso `tick_ms` (1 ms), il componente esegue in sequenza:

1. La disattivazione simultanea di tutte le 6 linee di selezione cifra (`selettori_cifra` $= 000000_2$) per prevenire sovrapposizioni luminose spurie (*ghosting*) durante il cambio dei segmenti.
2. La trascodifica combinatoria della parola celle_display[indice_scansione] (con indice_scansione $\in [0, 5] \rightarrow \lceil \log_2 6 \rceil =$ **3** bit) nei 7 bit di pilotaggio dei segmenti luminosi secondo la tabella di verità a 7 segmenti.
3. L'attivazione della sola linea `indice_scansione`-esima di `selettori_cifra` e l'incremento ciclico in aritmetica modulo 6: indice_scansione $\leftarrow$ (indice_scansione $+ 1$) mod 6.

Nome	Dimensione (bit)	Descrizione
`celle_display[0..5]`	24	Vettore di 6 parole da 4 bit contenente i simboli da mostrare (0–11)
`indice_scansione`	3	Contatore modulo 6 della cifra attiva nel multiplexing (intervallo 0–5)
`segmenti`	7	Linee logiche di pilotaggio dei 7 segmenti luminosi (G, F, E, D, C, B, A)
`selettori_cifra`	6	Linee di abilitazione selettiva delle 6 cifre (codifica 1-su-6)

1.2.8 Flusso logico complessivo

Il sistema complessivo evolve in modo concorrente e privo di attese bloccanti secondo la seguente pseudocodifica universale:

1. **Inizializzazione:** Poni stato = `STANDBY`, stato_filtro = `RIPOSO`, abilita_timer = 0, fine_tempo = 0, $M = 0$, $T = 15$, ultimo_tasto = 15, indice_scansione = 0 e poni tutte le 6 celle di `celle_display` a 11 (`VUOTO`).

2. **Ciclo di elaborazione:**

   - **2.1** Incrementa di un'unità il registro `seme_casuale`.
   - **2.2 (Cadenza 1 ms – Componente CLOCK):** Poni tick_ms = 1. Se abilita_timer == 1, incrementa l'accumulatore dei millisecondi; al raggiungimento di 1000, azzeralo e decrementa `secondi_residui`; se secondi_residui == 0, poni abilita_timer = 0 e fine_tempo = 1.
   - **2.3 (Cadenza 1 ms – Componente VISIVO):** Azzera `selettori_cifra`. Se stato $\neq$ `STANDBY`, decodifica celle_display[indice_scansione] su `segmenti`, attiva la linea `indice_scansione` di `selettori_cifra` e aggiorna indice_scansione $\leftarrow$ (indice_scansione + 1) mod 6.
   - **2.4 (Sincronismo tick_ms == 1 – Componenti SENSORI e CONTROLLO):**
     - Poni tick_ms = 0.
     - Esegui un passo dell'automa SENSORI aggiornando M e T.
     - Se fine_tempo == 1, poni fine_tempo = 0: se stato $\in$ {`ATTESA_CANCELLETTO`, `INSERIMENTO_OTP`} poni celle_display = `000000` e poni stato = `ALLARME`.
     - Se stato == `STANDBY`: se $M == 1$, poni $M = 0$, ultimo_tasto = 15, celle_display = `---` e poni stato = `ATTESA_ASTERISCO`.
     - Se $T \neq 15$, consuma il simbolo ($T_{\text{curr}} \leftarrow T$, $T \leftarrow 15$): se stato $\notin$ {`ALLARME`, `APERTO`} e (ultimo_tasto == $10 \wedge T_{\text{curr}} == 11$), poni abilita_timer = 0, celle_display = `VUOTO` e stato = `STANDBY`; altrimenti poni ultimo_tasto $\leftarrow T_{\text{curr}}$ e valuta le transizioni di stato (`ATTESA_ASTERISCO` $\rightarrow$ `INSERIMENTO_PIN` $\rightarrow$ `ATTESA_CANCELLETTO` $\rightarrow$ `INSERIMENTO_OTP` $\rightarrow$ `APERTO` / `ALLARME`).

3. **Ritorna al punto 2.**

1.3 Analisi della soluzione adottata

I cinque componenti individuati nella sezione di analisi (CLOCK, SENSORI, CONTROLLO, GENERATORE_OTP, VISIVO) sono stati progettati come moduli software indipendenti e concorrenti, coordinati dalle periferiche hardware di temporizzazione e interruzione del microcontrollore ATmega328P (operante alla frequenza di clock di 16 MHz).

1.3.1 Scelte comuni e architettura del sistema

1.3.1.1 Scelta della base di tempi e del Timer unico (Timer2)

Per soddisfare i requisiti temporali del progetto (scansione del display a 6 cifre senza sfarfallio, campionamento antirimbalzo della tastiera a 16 ms e timeout di sicurezza di 10 secondi per l'OTP) senza mai ricorrere ad attese bloccanti e impiegando il minor numero possibile di risorse hardware, l'intera gestione del tempo è stata unificata sul solo **Timer2 a 8 bit**, lasciando completamente liberi sia il Timer0 sia il Timer1.

Il Timer2 è configurato in modalità **CTC (Clear Timer on Compare Match – bit WGM21 = 1)**, nella quale il contatore hardware `TCNT2` incrementa automaticamente e si azzera non appena raggiunge il valore impostato nel registro di confronto `OCR2A`. Applicando un fattore di **prescaler** $N = 64$ **(bit CS22 = 1)**, la frequenza di incremento del timer risulta pari a 16 MHz/64 = 250 kHz (ovvero un passo ogni 4 μs). Per ottenere un interrupt esattamente ogni 1 ms ($f_{\text{target}} = 1000$ Hz, pari a 250 passi da 4 μs), il valore del registro `OCR2A` è calcolato secondo la formula del datasheet AVR:

$$\text{OCR2A} = \frac{f_{\text{clk_I/O}}}{N \cdot f_{\text{target}}} - 1 = \frac{16 \cdot 10^6}{64 \cdot 1000} - 1 = 250 - 1 = 249$$

In questo modo, l'interruzione hardware `TIMER2_COMPA_vect` scatta con precisione deterministica ogni 1 ms e governa simultaneamente tre flussi temporali:

1. **Sincronizzazione degli automi di foreground (`clk.tick`):** Ad ogni millisecondo alza il flag `clk.tick`, permettendo al ciclo principale di eseguire un singolo passo di scansione della tastiera e della macchina a stati centrale.

2. **Prescaler software per il timeout OTP (`clk.sec_left`):** Quando il timer OTP è attivo, conta blocchi di 1000 interruzioni da 1 ms (pari a 1 secondo esatto) e decrementa il registro dei secondi residui da 10 a 0.

3. **Multiplexing del display a 7 segmenti:** Ad ogni millisecondo commuta la cifra attiva tra le 6 disponibili. Ogni singola cifra viene rinfrescata ogni 6 ms, corrispondente a una frequenza di aggiornamento dell'intero display di 1000/6 $\approx$ 166,7 Hz, valore oltre tre volte superiore alla soglia di fusione ottica dell'occhio umano ($\sim$ 50 Hz).

1.3.1.2 Registri di I/O, variabili globali e comunicazione tra automi

In pieno rispetto del vincolo *bare-metal*, il programma rinuncia alle funzioni di astrazione di Arduino (`digitalRead`, `digitalWrite`, `shiftOut`, `delay`) e manipola direttamente gli *Special Function Registers* (SFR) dell'ATmega328P (`PORTB`, `PORTC`, `PINC`, `PORTD`, `PIND`, `DDRB`, `DDRC`, `DDRD`, `UCSR0A`, `UDR0`).

La comunicazione tra i moduli di background (ISR) e di foreground avviene esclusivamente attraverso quattro strutture dati globali (`clk`, `disp`, `kp`, `sys`) dichiarate con i qualificatori `volatile static`:

- `static`**:** Limita la visibilità delle strutture al singolo file sorgente, proteggendo lo spazio dei nomi e favorendo l'ottimizzazione degli indirizzi da parte del compilatore.

- `volatile`**:** Istruisce il compilatore `avr-gcc` sul fatto che tali variabili vengono modificate in modo asincrono all'interno delle routine di interruzione. Ciò impedisce al compilatore di memorizzare temporaneamente i valori nei registri interni della CPU (`r0-r31`), obbligandolo ad eseguire ad ogni accesso una lettura diretta dalla RAM (`LDS`) o una scrittura diretta in RAM (`STS`).

- **Atomicità delle operazioni a 8 bit:** Su un microcontrollore RISC a 8 bit come l'ATmega328P, la lettura o scrittura di una variabile a 16 bit richiederebbe due istruzioni consecutive da 8 bit (con il rischio che un interrupt scatti esattamente a metà tra i due byte, corrompendo il dato). Grazie al prescaler software a due stadi, tutte le variabili condivise tra ISR e foreground (incluso il contatore `clk.sec_left`) sono rappresentate su singoli byte (`uint8_t`): ogni accesso è pertanto **nativamente atomico** (compiuto in un'unica istruzione macchina non interrompibile), rendendo superflua la disabilitazione manuale degli interrupt nel ciclo principale.

1.3.2 Modulo CLOCK

Il modulo CLOCK è interamente contenuto nella `ISR(TIMER2_COMPA_vect)`. Oltre ad alzare il segnale `clk.tick = 1` ad ogni millisecondo, esso gestisce il conteggio alla rovescia dei 10 secondi tramite la variabile locale `static uint16_t ms_acc`:

- Quando il controllo accessi abilita il timer (clk.timer_en = 1), ad ogni interruzione da 1 ms l'accumulatore `ms_acc` viene incrementato di un'unità.
- Appena `ms_acc` raggiunge il valore 1000 (1 secondo trascorso), viene riazzerato e il contatore `clk.sec_left` viene decrementato di 1.
- Nell'istante in cui `clk.sec_left` tocca il valore 0, la ISR arresta automaticamente il timer (clk.timer_en = 0) e alza il flag di evento clk.expired = 1, segnalando al modulo CONTROLLO la scadenza del tempo massimo.
- Quando invece il timer è disabilitato (clk.timer_en == 0), la ISR mantiene costantemente `ms_acc = 0`: questa precauzione garantisce che, nel momento in cui un nuovo timer viene avviato, il conteggio del primo secondo parta esattamente da zero, evitando che un residuo precedente accorci la durata del primo secondo.

1.3.3 Modulo SENSORI (PIR e Tastiera)

Il modulo SENSORI acquisisce e pulisce i segnali provenienti dalle due periferiche di ingresso:

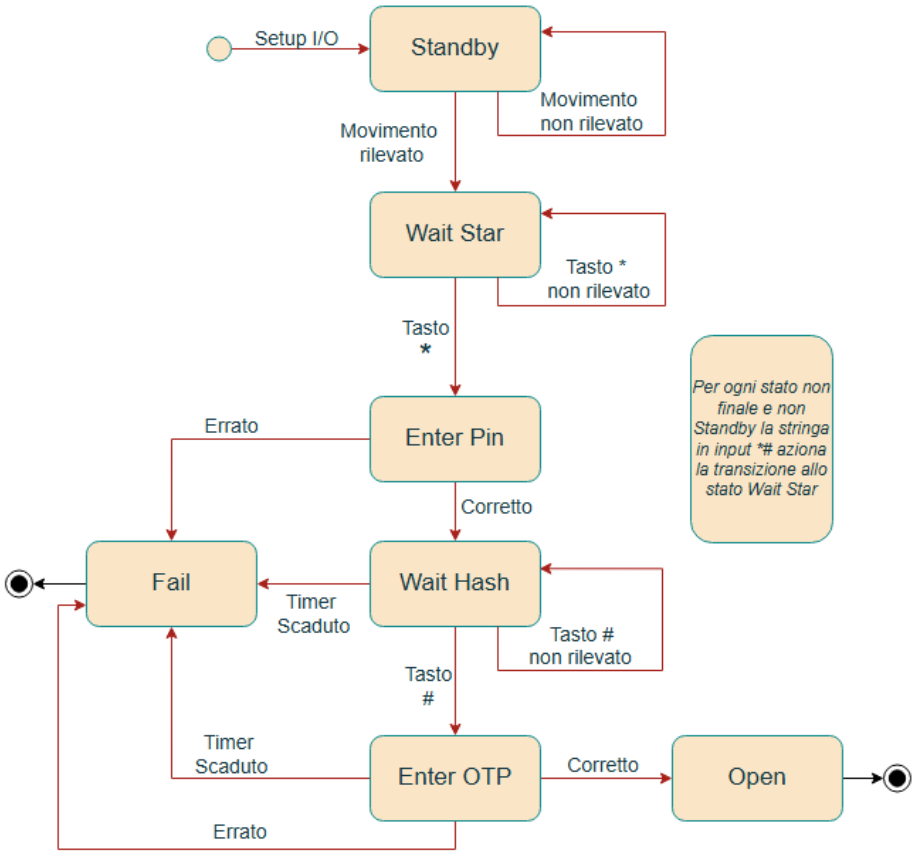
1. **Sensore di movimento PIR (Interrupt esterno `INT0` su pin PD2):** Per evitare di interrogare continuamente il sensore durante lo stato dormiente, l'uscita digitale del PIR è collegata al pin `PD2`, associato all'interruzione esterna hardware `INT0`. Configurando i bit ISC01 = 1 e ISC00 = 1 nel registro `EICRA` e abilitando il bit `INT0` in `EIMSK`, l'hardware solleva un interrupt nell'istante esatto del fronte di salita (0 V → 5 V). La relativa `ISR(INT0_vect)` imposta immediatamente sys.pir_flag = 1.
2. **Tastiera matriciale** 4×3 **(`scanKeys` e `runKeypad`):** Le 4 righe della tastiera sono connesse ai pin `PC0..PC3` (configurati come uscite digitali normalmente a livello logico alto 1), mentre le 3 colonne sono connesse ai pin `PC4`, `PC5` e `PD3` (configurati come ingressi digitali con resistori di *pull-up* interni attivi, che mantengono le colonne a livello alto 1 quando nessun tasto è premuto).
   - *Scansione elettrica (`scanKeys`):* Un ciclo itera sulle 4 righe ($r = 0..3$) portando a livello basso (0) una sola riga alla volta tramite `PORTC &= ~(1 << r)`. Dopo aver atteso 2 cicli di clock (`nop`, pari a 125 ns) per consentire la scarica delle capacità parassite dei fili e stabilizzare la tensione elettrica, la funzione legge i registri `PINC` e `PIND`: se una colonna $c \in [0, 2]$ si trova a livello basso (0), significa che il pulsante all'incrocio (r, c) è premuto e viene restituito l'indice lineare $3r + c \in [0, 11]$ (oppure `KEY_NONE = 0xFF` se al termine delle 4 righe nessun contatto risulta chiuso).

- *Macchina a stati di antirimbalzo (`runKeypad`):* Invocata in foreground ad ogni millisecondo, filtra i rimbalzi meccanici su 4 stati (`KP_IDLE`, `KP_VER_PRESS`, `KP_WAIT_REL`, `KP_VER_REL`). Una pressione viene validata e tradotta tramite la tabella `KEY_MAP` in `kp.key` solo quando la coordinata letta permane identica per 16 cicli consecutivi da 1 ms (16 ms netti). Successivamente l'automa attende che il tasto venga rilasciato stabilmente per ulteriori 16 ms prima di tornare in `KP_IDLE`, garantendo che una singola pressione prolungata dell'utente generi un solo evento logico.

1.3.4 Modulo CONTROLLO (FSM a 7 stati)

L'automa centrale `runSystem()` viene eseguito nel ciclo principale ad ogni millisecondo e coordina l'intera logica di sicurezza della cassaforte:

- In testa alla funzione vengono valutate due guardie prioritarie indipendenti dalla tastiera: la verifica di scadenza del tempo (clk.expired $\neq 0$, che porta immediatamente il sistema nello stato di allarme `ST_FAIL` se ci si trova in attesa dell'OTP) e la gestione dello stato dormiente `ST_STANDBY` (che attende esclusivamente l'evento sys.pir_flag $\neq 0$ per accendere i trattini sul display e passare in `ST_WAIT_STAR`).
- Se il sistema è attivo e il modulo tastiera ha depositato un nuovo simbolo in `kp.key` ($\neq$ `KEY_NONE`), l'automa preleva il simbolo e riazzera subito kp.key = `KEY_NONE` per consumare l'evento.
- Prima di elaborare il tasto nello stato corrente, se il sistema non è già bloccato negli stati terminali (`ST_FAIL` o `ST_OPEN`), il riconoscitore di sequenza controlla se il tasto precedente (`sys.last_key`) era `KEY_STAR` (`*`) e il tasto attuale è `KEY_HASH` (`#`): in caso affermativo disabilita il timer OTP, spegne i display e riporta immediatamente la macchina in `ST_STANDBY`.
- Infine, il costrutto `switch(sys.state)` fa evolvere la procedura attraverso le fasi di attesa asterisco (`ST_WAIT_STAR`), inserimento e verifica in serie del PIN (`ST_ENTER_PIN`), attesa cancelletto (`ST_WAIT_HASH`) e inserimento temporizzato dell'OTP (`ST_ENTER_OTP`).



La logica completa di transizione tra gli stati e le azioni eseguite sui registri condivisi è riassunta nella seguente tabella:

Stato Corrente	Condizione / Evento	Azioni Eseguite	Stato Succ.
ST_STANDBY	pir_flag == 1	Azzera pir_flag e last_key; setDisp(SEG_DASH)	ST_WAIT_STAR
ST_STANDBY	pir_flag == 0	Scarta eventuali tasti; display spento	ST_STANDBY
Stati non finali (WAIT_STAR, ENTER_PIN, WAIT_HASH, ENTER_OTP)	last_key == KEY_STAR ∧ k == KEY_HASH	Pone timer_en = 0, expired = 0, last_key = KEY_NONE; setDisp(SEG_BLANK)	ST_STANDBY
ST_WAIT_STAR	k == KEY_STAR	Pone idx = 0, code_ok = 1; setDisp(SEG_DASH)	ST_ENTER_PIN
ST_ENTER_PIN	$k \leq 9 \wedge$ idx < 5	disp.buf[idx] = k; se $k \neq$ SECRET_PIN[idx] pone code_ok = 0; idx++	ST_ENTER_PIN
ST_ENTER_PIN	$k \leq 9 \wedge$ idx == $5 \wedge$ code_ok == 1	setDisp(SEG_DASH); makeOtp(); pone sec_left = 10, expired = 0, timer_en = 1	ST_WAIT_HASH
ST_ENTER_PIN	$k \leq 9 \wedge$ idx == $5 \wedge$ code_ok == 0	setDisp(0) (Allarme 000000)	ST_FAIL
ST_WAIT_HASH	expired == 1	Azzera expired; setDisp(0)	ST_FAIL
ST_WAIT_HASH	k == KEY_HASH ∧ expired == 0	Pone idx = 0, code_ok = 1; setDisp(SEG_DASH)	ST_ENTER_OTP
ST_ENTER_OTP	expired == 1	Azzera expired; setDisp(0)	ST_FAIL
ST_ENTER_OTP	$k \leq 9 \wedge$ idx $< 5 \wedge$ expired == 0	disp.buf[idx] = k; se $k \neq$ otp[idx] pone code_ok = 0; idx++	ST_ENTER_OTP
ST_ENTER_OTP	$k \leq 9 \wedge$ idx == $5 \wedge$ code_ok == 1	Pone timer_en = 0; setDisp(1) (Apertura 111111)	ST_OPEN
ST_ENTER_OTP	$k \leq 9 \wedge$ idx == $5 \wedge$ code_ok == 0	Pone timer_en = 0; setDisp(0) (Allarme 000000)	ST_FAIL
ST_FAIL / ST_OPEN	Qualsiasi ingresso	Nessuna (Stati terminali bloccati)	ST_FAIL / ST_OPEN

1.3.5 Modulo GENERATORE_OTP, UART e Modulo VISIVO

1. **Calcolo OTP e Trasmissione UART0 (makeOtp e uartTx):** Nel ciclo while(1) del main, il registro a 16 bit sys.seed viene incrementato a frequenza libera (centinaia di kHz). Quando l'utente completa la digitazione del PIN valido, la funzione makeOtp() preleva sys.seed (impostandolo al valore di sicurezza 0xACE1 nel caso limite in cui valga 0) ed esegue per 6 volte i tre scorrimenti XOR dell'algoritmo Xorshift. Ciascuna cifra decimale ottenuta (0..9) viene salvata in sys.otp[i] e trasmessa sulla porta seriale UART0 (UBRR0L = 103, 9600 baud, formato 8N1) attendendo il bit UDRE0 (*USART Data Register Empty*) nel registro UCSR0A.

2. **Pilotaggio Shift Register 74HC595 e Multiplexing Display (srSend):** Per pilotare i 7 segmenti più gli anodi delle 6 cifre senza impiegare $7 + 6 = 13$ pin del microcontrollore, i catodi $A..G$ dei display sono collegati alle uscite parallele $Q_0..Q_6$ di uno Shift Register 74HC595, controllato tramite soli 3 pin su PORTD: PD4 (DS, ingresso dati seriale), PD5 (SHCP,

clock di scorrimento dei Flip-Flop interni) e `PD6` (`STCP`, clock del registro di Latch di uscita). Ad ogni millisecondo, la `ISR(TIMER2_COMPA_vect)` spegne tutti i 6 anodi su `PORTB` (`PORTB &= ~0x3F`), preleva la maschera a 7 segmenti della cifra corrente d dalla tabella `SEG_MAP` e la inverte bit a bit con l'operatore `~` (poiché i display sono ad **anodo comune** e quindi un segmento si accende portando il suo catodo a 0 V). La funzione `srSend()` invia i singoli 8 bit sul pin `PD4` generando 8 fronti di clock su `PD5`; solo al termine degli 8 bit emette un impulso sul pin di Latch `PD6`, aggiornando istantaneamente tutte le uscite parallele senza sfarfallii intermedi, e infine alimenta l'anodo della cifra d tramite `PORTB |= (1 << d)`.

1.3.6 Sistema complessivo

Il sistema complessivo adotta un'architettura sincrona e concorrente così articolata:

1. Al boot, `main()` invoca le primitive `initIO()`, `initUART()`, `initBaseTempi()` e `initPIR()`, abilita globalmente le interruzioni ponendo a 1 il bit 7 del registro di stato `SREG` ed entra nel ciclo infinito `while(1)`.

2. Le ISR `INT0_vect` e `TIMER2_COMPA_vect` operano in background garantendo la cattura immediata del movimento, la generazione della base tempi a 1 ms, il decremento a 1 s del timeout OTP e la scansione continua in multiplexing delle 6 cifre del display.

3. Il ciclo foreground incrementa ininterrottamente il seme di entropia `sys.seed` e, ad ogni scadenza del flag `clk.tick`, esegue in pochi microsecondi e senza blocchi gli automi `runKeypad()` e `runSystem()`.

2 Sintesi

2.1 Sintesi Modulo CLOCK e Base Tempi

Il Timer2 a 8 bit è configurato in modalità CTC per generare un'interruzione ogni 1 ms (1000 Hz). La struttura dati `Clock` contiene esclusivamente campi atomici a 8 bit (`uint8_t`):

```
typedef struct {
    uint8_t tick;       // Flag cadenza 1 ms per scansione foreground (1 bit)
    uint8_t timer_en;   // Abilitazione countdown OTP (1 bit)
    uint8_t expired;    // Flag scadenza 10 secondi (1 bit)
    uint8_t sec_left;   // Secondi residui (4 bit effettivi: 0..10)
} Clock;

volatile static Clock clk = {0, 0, 0, 0};

static inline void initBaseTempi(void) {
    TCCR2A = (1 << WGM21);  // Seleziona modalita CTC (WGM21 = 1)
    TCCR2B = (1 << CS22);   // Imposta prescaler a 64 (CS22 = 1)
    OCR2A  = 249;           // Limite contatore per periodo esatto di 1 ms
    TIMSK2 = (1 << OCIE2A); // Abilita interrupt su Compare Match A
}

ISR(TIMER2_COMPA_vect) {
    static uint16_t ms_acc = 0; // Accumulatore locale per base tempi 1 secondo

    // 1. Poni a 1 il flag di sincronismo a 1 ms per il foreground
    clk.tick = 1;

    // 2. Se il timer OTP e attivo, allora aggiorna il prescaler a 1 secondo
    if (clk.timer_en != 0) {
        ms_acc++;
        // Se sono trascorsi 1000 ms (1 s), allora decrementa i secondi residui
        if (ms_acc >= 1000) {
            ms_acc = 0;
            if (clk.sec_left > 0) {
                clk.sec_left--;
                // Se i secondi residui raggiungono 0, allora segnala scadenza
                if (clk.sec_left == 0) {
                    clk.timer_en = 0;
                    clk.expired  = 1;
                }
            }
        }
    } else {
        // Altrimenti mantieni azzerato l'accumulatore per allineare la fase
        ms_acc = 0;
    }
    // ... (Segue multiplexing display)
}
```

- `TCCR2A = (1 << WGM21);` Seleziona la modalità CTC; il contatore `TCNT2` si azzera non appena eguaglia `OCR2A`.
- `TCCR2B = (1 << CS22)` e `OCR2A = 249;` Con prescaler 64 (1 tick = 4 μs), il conteggio di 250 passi (0..249) produce un periodo esatto di 1000 μs = 1 ms.
- `static uint16_t ms_acc;` Dichiarato `static` all'interno della ISR per preservare il valore tra le interruzioni mantenendo la visibilità strettamente locale.

2.2 Sintesi Modulo SENSORI (PIR e Tastiera)

Il modulo gestisce il rilevatore PIR tramite l'interrupt esterno `INT0` e la tastiera matriciale tramite scansione per righe e FSM di antirimbalzo a 4 stati:

```
typedef enum {
    KP_IDLE      = 0,
    KP_VER_PRESS = 1,
    KP_WAIT_REL  = 2,
    KP_VER_REL   = 3
} KpState;

typedef struct {
    KpState state;      // Stato FSM antirimbalzo (2 bit)
    uint8_t raw;        // Coordinata grezza 3*r + c (4 bit)
    uint8_t stable;     // Coordinata in verifica (4 bit)
    uint8_t ms_cnt;     // Contatore campioni stabili (5 bit: 0..16)
    uint8_t key;        // Tasto validato a 4 bit (0..11, 0xFF=nullo)
} Keypad;

volatile static Keypad kp = {KP_IDLE, KEY_NONE, KEY_NONE, 0, KEY_NONE};

static inline void initPIR(void) {
    EICRA = (1 << ISC01) | (1 << ISC00); // Fronte di salita su INT0 (PD2)
    EIMSK = (1 << INT0);                 // Abilita interruzione INT0
}

ISR(INT0_vect) {
    sys.pir_flag = 1; // Se rilevato fronte di salita PIR, allora alza il flag
}

static inline uint8_t scanKeys(void) {
    for (uint8_t r = 0; r < 4; r++) {
        PORTC |= 0x0F;                  // Poni tutte le righe PC0..PC3 a livello
    ALTO
        PORTC &= ~(1 << r);             // Poni la sola riga r-esima a livello
    BASSO
        asm volatile("nop\n\tnop\n\t"); // Attendi 125 ns per assestamento linee

        // Se una colonna legge livello BASSO (0), allora restituisci indice 3*r +
     c
        if ((PINC & (1 << PINC4)) == 0) return (uint8_t)(r * 3 + 0);
        if ((PINC & (1 << PINC5)) == 0) return (uint8_t)(r * 3 + 1);
        if ((PIND & (1 << PIND3)) == 0) return (uint8_t)(r * 3 + 2);
    }
    PORTC |= 0x0F;
    return KEY_NONE; // Altrimenti restituisci codice di nessun tasto premuto
}

static inline void runKeypad(void) {
    uint8_t c = scanKeys();
    kp.raw = c;

    switch (kp.state) {
        case KP_IDLE:
            // Se viene rilevato un contatto chiuso, allora avvia verifica
    pressione
            if (c != KEY_NONE) {
                kp.stable = c;
                kp.ms_cnt = 0;
                kp.state  = KP_VER_PRESS;
            }
            break;
```

```
        case KP_VER_PRESS:
            // Se la coordinata resta uguale a quella salvata, allora incrementa
    contatore
            if (c == kp.stable) {
                kp.ms_cnt++;
                // Se il contatto e stabile da 16 ms, allora emetti il tasto
    validato
                if (kp.ms_cnt >= DEBOUNCE_MS) {
                    kp.key   = KEY_MAP[c];
                    kp.state = KP_WAIT_REL;
                }
            } else {
                // Altrimenti (rimbalzo spurio) torna in stato di riposo
                kp.state = KP_IDLE;
            }
            break;

        case KP_WAIT_REL:
            // Se il tasto viene rilasciato, allora avvia verifica di rilascio
            if (c == KEY_NONE) {
                kp.ms_cnt = 0;
                kp.state  = KP_VER_REL;
            }
            break;

        case KP_VER_REL:
            // Se l'assenza di contatto permane stabile per 16 ms, allora torna in
     IDLE
            if (c == KEY_NONE) {
                kp.ms_cnt++;
                if (kp.ms_cnt >= DEBOUNCE_MS) {
                    kp.state = KP_IDLE;
                }
            } else {
                // Altrimenti (rimbalzo in apertura) resta in attesa di rilascio
                kp.state = KP_WAIT_REL;
            }
            break;
    }
}
```

2.3 Sintesi Modulo CONTROLLO

Il modulo implementa la macchina a stati finiti a 7 stati che governa l'autenticazione a due fattori, il controllo di scadenza dei 10 secondi e il reset rapido *#:

```
typedef enum {
    ST_STANDBY   = 0,
    ST_WAIT_STAR = 1,
    ST_ENTER_PIN = 2,
    ST_WAIT_HASH = 3,
    ST_ENTER_OTP = 4,
    ST_FAIL      = 5,
    ST_OPEN      = 6
} SysState;

typedef struct {
    SysState state;     // Stato FSM centrale (3 bit)
    uint8_t  pir_flag; // Flag fronte di salita sensore PIR (1 bit)
    uint8_t  idx;       // Indice cifra in inserimento (3 bit: 0..6)
    uint8_t  code_ok;   // Accumulatore booleano verifica in serie (1 bit)
    uint8_t  last_key; // Memoria tasto precedente per reset *# (4 bit)
    uint8_t  otp[6];    // Cifre OTP in binario puro (6 x 4 bit: 0..9)
    uint16_t seed;      // Contatore entropia a 16 bit per Xorshift
} System;

volatile static System sys = {ST_STANDBY, 0, 0, 1, KEY_NONE, {0, 0, 0, 0, 0, 0}, 0
    xACE1};

static inline void runSystem(void) {
    // 1. Se il timer di 10s e scaduto durante attesa o inserimento OTP, allora
    vai in FAIL
    if (clk.expired != 0) {
        clk.expired = 0;
        if (sys.state == ST_WAIT_HASH || sys.state == ST_ENTER_OTP) {
            setDisp(0);
            sys.state = ST_FAIL;
            return;
        }
    }

    // 2. Se il sistema e in STANDBY e il PIR rileva movimento, allora passa in
    WAIT_STAR
    if (sys.state == ST_STANDBY) {
        if (sys.pir_flag != 0) {
            sys.pir_flag = 0;
            sys.last_key = KEY_NONE;
            setDisp(SEG_DASH);
            sys.state    = ST_WAIT_STAR;
        }
        kp.key = KEY_NONE;
        return;
    }
    sys.pir_flag = 0;

    // 3. Se non vi e alcun nuovo tasto premuto, allora termina il ciclo di
    scansione
    uint8_t k = kp.key;
    if (k == KEY_NONE) return;
    kp.key = KEY_NONE;

    // 4. Se in stato non finale viene premuto '*' seguito da '#', allora resetta
    in STANDBY
    if (sys.state != ST_FAIL && sys.state != ST_OPEN) {
        if (sys.last_key == KEY_STAR && k == KEY_HASH) {
```

```
            clk.timer_en = 0;
            clk.expired  = 0;
            sys.last_key = KEY_NONE;
            setDisp(SEG_BLANK);
            sys.state    = ST_STANDBY;
            return;
        }
        sys.last_key = k;
    }

    // 5. Evoluzione Macchina a Stati Finiti Centrale
    switch (sys.state) {
        case ST_WAIT_STAR:
            // Se viene premuto '*', allora azzera contatore e passa a inserimento
     PIN
            if (k == KEY_STAR) {
                sys.idx     = 0;
                sys.code_ok = 1;
                setDisp(SEG_DASH);
                sys.state   = ST_ENTER_PIN;
            }
            break;

        case ST_ENTER_PIN:
            // Se il tasto e una cifra (0..9), allora mostrala e verifica in serie
     col PIN
            if (k <= 9) {
                uint8_t i = sys.idx;
                disp.buf[i] = k;
                // Se la cifra non coincide col PIN segreto, allora poni code_ok =
    0
                if (k != SECRET_PIN[i]) {
                    sys.code_ok = 0;
                }
                i++;
                sys.idx = i;

                // Se sono state inserite tutte le 6 cifre, allora valuta l'esito
                if (i == 6) {
                    // Se tutte le 6 cifre sono corrette, allora genera OTP e
    avvia timer 10s
                    if (sys.code_ok != 0) {
                        setDisp(SEG_DASH);
                        makeOtp();
                        clk.sec_left = OTP_TIMEOUT_S;
                        clk.expired  = 0;
                        clk.timer_en = 1;
                        sys.state    = ST_WAIT_HASH;
                    } else {
                        // Altrimenti attiva blocco di allarme (000000)
                        setDisp(0);
                        sys.state = ST_FAIL;
                    }
                }
            }
            break;

        case ST_WAIT_HASH:
            // Se viene premuto '#' entro i 10s, allora passa a inserimento OTP
            if (k == KEY_HASH) {
                sys.idx     = 0;
                sys.code_ok = 1;
                setDisp(SEG_DASH);
```

```
                sys.state    = ST_ENTER_OTP;
            }
            break;

        case ST_ENTER_OTP:
            // Se il tasto e una cifra (0..9), allora mostrala e verifica in serie
     con l'OTP
            if (k <= 9) {
                uint8_t i = sys.idx;
                disp.buf[i] = k;
                // Se la cifra non coincide con l'OTP generato, allora poni
    code_ok = 0
                if (k != sys.otp[i]) {
                    sys.code_ok = 0;
                }
                i++;
                sys.idx = i;

                // Se e stata inserita la sesta cifra dell'OTP, allora ferma il
    timer e valuta
                if (i == 6) {
                    clk.timer_en = 0;
                    // Se l'OTP e corretto, allora sblocca cassaforte (111111),
    altrimenti allarme
                    if (sys.code_ok != 0) {
                        setDisp(1);
                        sys.state = ST_OPEN;
                    } else {
                        setDisp(0);
                        sys.state = ST_FAIL;
                    }
                }
            }
            break;

        default:
            break;
    }
}
```

2.4 Sintesi Modulo GENERATORE_OTP, UART e Modulo VISIVO

La serializzazione degli 8 bit verso lo shift register `74HC595` (`srSend`), la generazione pseudo-casuale Xorshift (`makeOtp`) e la trasmissione seriale (`uartTx`) sono implementate tramite primitive `static inline`:

```
static inline void srSend(uint8_t val) {
    for (uint8_t i = 0; i < 8; i++) {
        // Se il bit piu significativo (bit 7) vale 1, allora poni D4 (DS) ALTO,
    altrimenti BASSO
        if ((val & 0x80) != 0) {
            PORTD |= (1 << 4);  // D4 (DS) HIGH (Istruzione SBI)
        } else {
            PORTD &= ~(1 << 4); // D4 (DS) LOW  (Istruzione CBI)
        }
        PORTD |= (1 << 5);      // Impulso di salita su D5 (SHCP Clock)
        PORTD &= ~(1 << 5);     // Impulso di discesa su D5 (SHCP Clock)
        val <<= 1;              // Scorri a sinistra per preparare il bit
    successivo
    }
    PORTD |= (1 << 6);          // Fronte alto su D6 (STCP Latch) per aggiornare
    le uscite
    PORTD &= ~(1 << 6);         // Fronte basso su D6 (STCP Latch)
}

static inline void uartTx(uint8_t b) {
    // Finche il buffer di trasmissione non e vuoto (UDRE0 == 0), attendi
    while ((UCSR0A & (1 << UDRE0)) == 0);
    UDR0 = b; // Carica il byte nel registro dati UART
}

static inline void makeOtp(void) {
    uint16_t s = sys.seed;
    // Se il seme campionato vale 0, allora imposta valore non nullo di sicurezza
    if (s == 0) {
        s = 0xACE1;
    }
    for (uint8_t i = 0; i < 6; i++) {
        s ^= (s << 7);
        s ^= (s >> 9);
        s ^= (s << 8);
        uint8_t d = (uint8_t)(s & 0x0F);
        // Se il valore a 4 bit supera 9, allora sottrai 6 per ricondurlo in range
     4..9
        if (d > 9) {
            d -= 6;
        }
        sys.otp[i] = d;
        uartTx(d + 48); // Invia carattere convertito in ASCII ('0' = 48)
    }
    uartTx(13); // Invia Carriage Return (CR)
    uartTx(10); // Invia Line Feed (LF)
}
```

2.5 Analisi di livello MV2 e qualità del codice assembly

La scrittura bare-metal controlla direttamente la traduzione del codice C in linguaggio Assembly AVR (livello MV2), riducendo al minimo l'impiego di registri e cicli di clock.

2.5.1 1. Ottimizzazione delle istruzioni di I/O (`SBI`, `CBI`, `SBIC`, `SBIS`)

L'accesso diretto ai registri di periferica (`PORTB`, `PORTC`, `PINC`, `PORTD`, `PIND`, `UCSR0A`) genera le istruzioni native a singolo bit del set RISC AVR a 8 bit:

- Le impostazioni dei bit di uscita (`PORTD |= (1 « 4)`, `PORTD |= (1 « 5)`, `PORTD |= (1 « 6)`) vengono tradotte nell'istruzione `SBI` (*Set Bit in I/O Register*), eseguita in **2 cicli di clock**.
- Gli azzeramenti (`PORTD &= ~(1 « 4)`, `PORTD &= ~(1 « 5)`) vengono convertiti nell'istruzione `CBI` (*Clear Bit in I/O Register*, **2 cicli di clock**).
- La lettura delle colonne della tastiera (`(PINC & (1 « PINC4)) == 0`) e del flag seriale (`(UCSR0A & (1 « UDRE0)) == 0`) viene compilata con le istruzioni di salto su bit `SBIC` (*Skip if Bit in I/O Register is Cleared*) e `SBIS` (*Skip if Bit in I/O Register is Set*), che valutano lo stato del pin in **1 o 2 cicli di clock** senza occupare registri di lavoro (`r0-r31`).

2.5.2 2. Effetto architetturale di `static inline`

Tutte le funzioni di interfacciamento hardware e gli automi invocati nel ciclo principale sono dichiarati `static inline`. A livello MV2, il compilatore innesta direttamente il corpo delle istruzioni nel punto di chiamata:

- Vengono eliminate le istruzioni di chiamata (`CALL`/`RCALL`, 3–4 cicli) e di ritorno (`RET`, 4 cicli).
- Si evita l'occupazione dello Stack in SRAM e il tempo associato alle istruzioni `PUSH` e `POP` di salvataggio del contesto.

2.5.3 3. Impatto del qualificatore `volatile` e atomicità a 8 bit

Il qualificatore `volatile` applicato alle strutture condivise (`clk`, `disp`, `kp`, `sys`) impone al compilatore `avr-gcc` di eseguire per ogni lettura un'istruzione `LDS` (*Load Direct from Data Space*, 2 cicli) e per ogni scrittura un'istruzione `STS` (*Store Direct to Data Space*, 2 cicli), garantendo la coerenza tra le ISR e il foreground. Inoltre, la riduzione del contatore `clk.sec_left` a 8 bit (`uint8_t`) tramite il prescaler locale a 1 secondo rende ogni operazione di avvio o arresto del timer nativamente atomica.

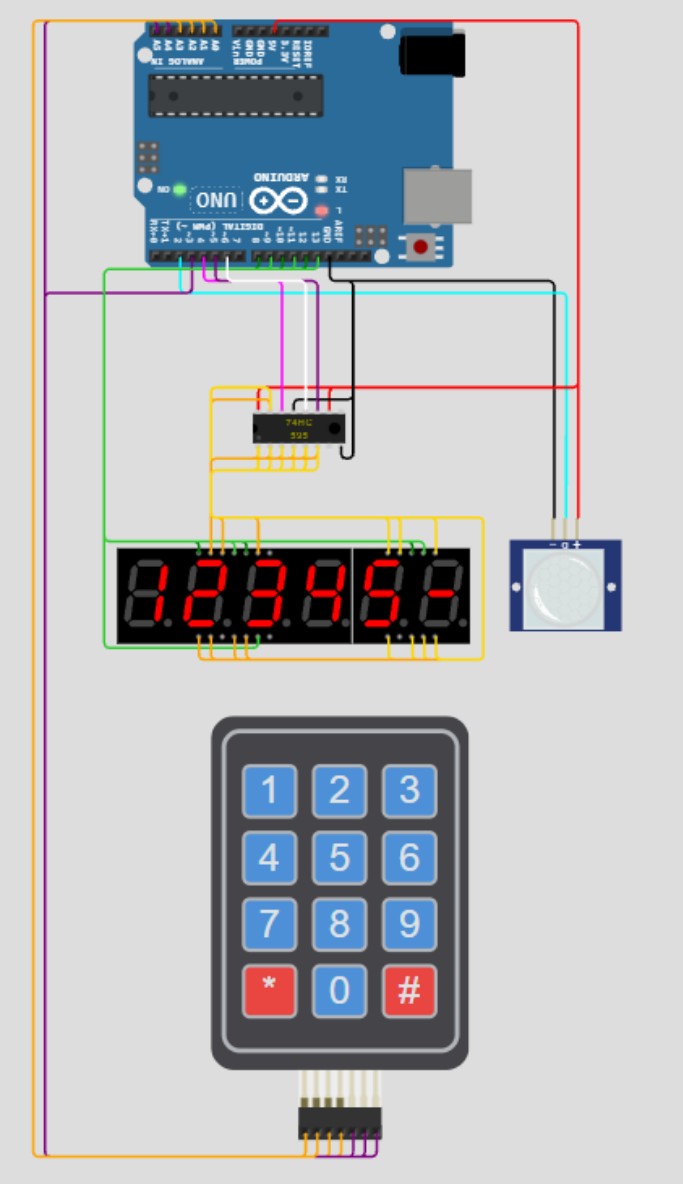
2.5.4 4. Costo computazionale delle ISR

- `ISR(INT0_vect)`: Esegue una sola istruzione `LDI` e una `STS` per porre sys.pir_flag = 1, concludendosi in meno di **15 cicli di clock** ($\sim 0{,}9$ **μs a 16 MHz)**.
- `ISR(TIMER2_COMPA_vect)`: Esegue l'aggiornamento di `clk.tick`, l'incremento di `ms_acc` e la funzione inlinata `srSend()` per il multiplexing visivo, attestandosi sotto i **130 cicli di clock** (~ 8 **μs a 16 MHz)**, pari a meno dello **0,8%** della capacità computazionale della CPU.

3 Sistema complessivo

Il sistema realizzato è disponibile e simulabile al seguente indirizzo:

https://wokwi.com/projects/478132265244059649



La topologia del circuito interconnette i seguenti moduli fisici:

- **Unità di elaborazione:** ATmega328P (Arduino Uno) a 16 MHz.
- **Sensore di movimento (`pir1`):** Uscita digitale cablata sul pin **D2** (`PD2 / INT0`).
- **Tastiera matriciale** 4×3 **(`keypad1`):** Righe `R1..R4` connesse ai pin **A0..A3** (`PC0..PC3`); colonne `C1..C3` connesse ai pin **A4** (`PC4`), **A5** (`PC5`) e **D3** (`PD3`).
- **Shift Register (`sr1` – 74HC595):** Pilotato tramite i pin **D4** (`PD4` – **DS**), **D5** (`PD5` – **SHCP**) e **D6** (`PD6` – **STCP**), con uscite `Q0..Q6` collegate ai catodi `A..G` dei display.
- **Display a 7 segmenti (`sevseg1` e `sevseg2`):** Due moduli ad anodo comune da 4 e 2 cifre (6 cifre totali), con anodi comuni `DIG1..DIG6` connessi ai pin **D8..D13** (`PB0..PB5`).

3.1 Manuale di utilizzo

Per collaudare e verificare il funzionamento del sistema sul simulatore Wokwi, avviare l'emulazione ed eseguire le seguenti verifiche:

1. **Avvio in stato dormiente e Risveglio tramite PIR**
   - *Azione:* Avviare la simulazione. Verificare che il display a 6 cifre sia spento (`ST_STANDBY`). Cliccare sul sensore PIR (`pir1`) e premere **"Simulate motion"**.
   - *Comportamento atteso:* L'interrupt `INT0` risveglia la centralina, che accende i 6 display mostrando i trattini centrali (`---`) e commuta in `ST_WAIT_STAR`.
2. **Inserimento e Verifica del Codice Personale PIN (1° Fattore)**
   - *Azione:* Premere il tasto `*` sul tastierino e digitare le 6 cifre del PIN autorizzato: `1 2 3 4 5 6`.
   - *Comportamento atteso:* Alla pressione di `*` il sistema entra in `ST_ENTER_PIN`. Ogni cifra premuta compare in sequenza sul display. Completata la sesta cifra corretta, il sistema ripristina `---` sul display, calcola e stampa sul *Serial Monitor* il codice OTP a 6 cifre, avvia il countdown di 10 secondi e transita in `ST_WAIT_HASH`. (Se viene inserito un PIN errato, il display mostra `000000` bloccando il sistema in `ST_FAIL`).
3. **Conferma e Inserimento temporizzato dell'OTP (2° Fattore)**
   - *Azione:* Entro 10 secondi dalla stampa dell'OTP, premere il tasto `#` e digitare le 6 cifre dell'OTP lette sul terminale seriale.
   - *Comportamento atteso:* Alla pressione di `#` l'automa passa in `ST_ENTER_OTP`. Completando le 6 cifre esatte entro i 10 secondi, il timer si arresta, il display mostra `111111` e la centralina raggiunge lo stato di apertura cassaforte (`ST_OPEN`).
4. **Simulazione di Allarme per Scadenza Tempo (Timeout 10s) o OTP Errato**
   - *Azione:* Generato l'OTP, lasciare trascorrere più di 10 secondi senza completare la digitazione (oppure inserire una cifra errata nell'OTP).
   - *Comportamento atteso:* Allo scadere del decimo secondo (o della sesta cifra errata), il sistema commuta in `ST_FAIL` mostrando `000000`.
5. **Interruzione volontaria tramite sequenza `*#` (Override Reset)**
   - *Azione:* Durante qualsiasi stato operativo non terminale (`ST_WAIT_STAR`, `ST_ENTER_PIN`, `ST_WAIT_HASH`, `ST_ENTER_OTP`), premere in sequenza il tasto `*` seguito dal tasto `#`.
   - *Comportamento atteso:* Il sistema arresta il timer dei 10 secondi, azzera i contatori, spegne i 6 display e ritorna nello stato dormiente `ST_STANDBY`.

3.2 Considerazioni finali

Il risultato principale della soluzione progettata risiede nell'azzeramento totale delle attese bloccanti e nella minimizzazione delle risorse hardware e di memoria impiegate:

- **Base tempi deterministica e prescaler a 8 bit:** Il Timer2 in modalità CTC genera un'interruzione esatta ogni 1 ms, cadenzando in background il multiplexing dei 6 display, il campionamento antirimbalzo a 16 ms della tastiera e il prescaler a 1 s per il countdown atomico a 8 bit (`sec_left`) dell'OTP, lasciando completamente liberi il Timer0 e il Timer1.
- **Verifica booleana in serie a memoria minima:** L'accumulatore a 1 bit `code_ok` aggiornato cifra per cifra e la codifica binaria diretta a 4 bit dei tasti hanno eliminato i buffer di input in SRAM e le tabelle di conversione ASCII interne.
- **Efficienza MV2:** L'uso delle dichiarazioni `static inline`, del qualificatore `volatile` e delle manipolazioni dirette sui registri di porta ha permesso al compilatore `avr-gcc` di tradurre gli accessi a I/O e RAM in istruzioni assembly da 1–2 cicli di clock (`SBI`, `CBI`, `SBIC`, `SBIS`, `LDS`, `STS`), mantenendo il carico della ISR sotto lo 0,8% del tempo CPU.